TEORÍA – DESARROLLO DE UN MICROSERVICIO DE PRODUCTOS CON YII

1. Introducción

Actualmente las aplicaciones modernas requieren arquitecturas que permitan crecer, mantenerse y desplegar componentes de manera independiente. Una de las arquitecturas más utilizadas es la **arquitectura de microservicios**, donde una aplicación se divide en pequeños servicios especializados que se comunican entre sí mediante protocolos como HTTP.

En este laboratorio se implementa un **microservicio de productos utilizando Yii2**, exponiendo funcionalidades mediante una **API REST** que permite realizar operaciones CRUD (Crear, Consultar, Actualizar y Eliminar).

---

2. Microservicios

Un **microservicio** es una unidad de software independiente diseñada para cumplir una función específica dentro de un sistema.

Cada microservicio:

- Tiene una responsabilidad concreta.
- Puede desplegarse independientemente.
- Posee su propia lógica de negocio.
- Se comunica mediante APIs.

Ejemplo:

Sistema E-commerce

Cliente
↓
API Gateway
↓
———————————————

| Usuarios Service |
| Productos Service |
| Ventas Service |

Pagos Service

En este laboratorio se desarrolla únicamente:

Microservicio de Productos

Responsabilidades:

- Registrar productos
- Consultar productos
- Actualizar productos
- Eliminar productos

---

3. Arquitectura REST

REST (**Representational State Transfer**) es un estilo arquitectónico utilizado para construir servicios web.

REST trabaja mediante:

- Recursos
- Métodos HTTP
- Respuestas generalmente en JSON

Ejemplo:

Método	Endpoint	Acción
GET	/productos	Listar
GET	/productos/1	Consultar
POST	/productos	Crear
PUT	/productos/1	Actualizar
DELETE	/productos/1	Eliminar

Ejemplo de respuesta JSON:

{

"id":1,
"nombre":"Laptop",
"precio":3500
}

---

4. Framework Yii2

Yii (**Yes It Is!**) es un framework de desarrollo web basado en PHP que sigue el patrón **MVC (Modelo–Vista–Controlador)**.

Características principales:

- Alto rendimiento.
- Integración con bases de datos.
- Soporte REST.
- Validaciones automáticas.
- ActiveRecord.

Arquitectura simplificada:

Cliente
↓
Controller
↓
Model
↓
Base de Datos

---

5. Patrón MVC en Yii

Modelo (Model)

Representa los datos y reglas del negocio.

Ejemplo:

class Producto extends ActiveRecord

Funciones:

- Validar datos.
- Conectarse a BD.
- Ejecutar consultas.

Controlador (Controller)

Recibe solicitudes HTTP.

Ejemplo:

class ProductoController
extends ActiveController

Funciones:

- Procesar solicitudes.
- Invocar modelos.
- Retornar JSON.

Vista (View)

En microservicios REST normalmente no se utilizan vistas.

Salida:

{
"id":1,
"nombre":"Laptop"
}

---

6. Composer

Composer es el administrador de dependencias de PHP.

Permite:

- Instalar Yii.
- Actualizar paquetes.
- Gestionar librerías.

Comando utilizado:

composer create-project
yiisoft/yii2-app-basic
productos-service

Proceso:

Composer
↓
Descarga paquetes
↓
Instala dependencias
↓
Genera proyecto

---

7. Migraciones en Yii

Las migraciones permiten administrar cambios de la base de datos mediante código.

Ventajas:

- Control de versiones.
- Repetibilidad.
- Automatización.

Ejemplo:

php yii migrate

Proceso:

Migración
↓
Crear tabla
↓
Actualizar BD

Ejemplo:

```
$this->createTable(
'producto'
);
```

---

8. ActiveRecord

ActiveRecord es un patrón que permite representar tablas como objetos.

Ejemplo:

Tabla:

producto

Objeto:

Producto::find()

Consulta:

Producto::findOne(1);

Inserción:

$model->save();

Beneficios:

- Reduce SQL manual.
- Facilita mantenimiento.

---

9. API REST en Yii

Yii incorpora soporte REST mediante:

yii\rest\ActiveController

Este controlador genera automáticamente:

index()
view()

create()
update()
delete()

Equivalencia:

Método HTTP	Función
GET	index
GET ID	view
POST	create
PUT	update
DELETE	delete

---

10. Formato JSON

JSON (**JavaScript Object Notation**) es el formato estándar para intercambio de datos entre sistemas.

Ejemplo:

```
{
"nombre":"Laptop",
"stock":20
}
```

Ventajas:

- Ligero.
- Fácil lectura.
- Compatible con múltiples lenguajes.

---

11. Flujo del laboratorio

Cliente
↓

HTTP Request
↓
ProductoController
↓
Producto Model
↓
MySQL
↓
JSON Response

Ejemplo:

POST /productos
↓
Validación
↓
Guardar
↓
Respuesta JSON

---

12. Conclusión

El desarrollo de un microservicio con Yii permite implementar una arquitectura desacoplada basada en servicios independientes. Mediante el uso de REST, ActiveRecord y migraciones se logra construir una API mantenible, escalable y preparada para integrarse con otros sistemas.